

Building a Todo App in React — Student Manual

Integrative Programming II

How to Use This Manual

This manual is broken into **lessons**. Each lesson:

1. Has **one new concept** to learn (don't skip ahead).
2. Gives you **step-by-step instructions** with code.
3. Ends with a **Checkpoint** — your app should look/behave a specific way.
4. Ends with **Reflection Questions** you must answer in your own words before moving to the next lesson.

Reflection Rule: After finishing each lesson, write your reflection in a one (1) whole yellow pad with your **NAME**, **SECTION**, and **DATE** written on it. A reflection is NOT a summary of this manual — it is ****your own explanation****, in your own words, of what the concept does and why it exists. One-line answers like "state manages data" will not be accepted.

You will build **one app** across all lessons: a Todo List application. By the end, it will have multiple pages, add/complete/delete functionality, filtering, and it will remember your todos even after refreshing the browser.

Prerequisites

- Node.js installed (check with `node -v` in your terminal)
 - A code editor (VS Code recommended)
 - Basic JavaScript (variables, functions, arrays, objects)
-

Lesson 0 — Project Setup & Folder Structure

Objective

Create a working React project using Vite and understand what every folder is for.

Steps

1. Open your terminal and run:

```
npm create vite@latest todo-app -- --template react
cd todo-app
npm install
npm run dev
```

2. Open the link shown in the terminal (usually `http://localhost:5173`). You should see the default Vite + React starter page.
3. Open the project folder in VS Code and inspect the structure:

```
todo-app/
├── public/           # static files, untouched by build tools
├── src/
│   ├── assets/      # images, global CSS
│   ├── components/  # (create this) reusable UI pieces
│   ├── pages/       # (create this) full page views
│   ├── App.jsx      # root component
│   └── main.jsx      # entry point — mounts App into the DOM
├── index.html
└── package.json
```

4. Create two new folders inside `src/`: `components` and `pages`. You will use these starting in Lesson 2.
5. Delete the boilerplate content inside `App.jsx` and `App.css` — replace `App.jsx` with:

```
function App() {
  return (
    <div className="app">
      <h1>My Todo App</h1>
    </div>
  );
}

export default App;
```

Checkpoint

Your browser shows "My Todo App" and nothing else. No errors in the terminal or browser console.

Reflection Questions

- What is the difference between `public/` and `src/`? Why can't you just put everything in one folder?
 - What does `main.jsx` do, and why does `App.jsx` need to be imported into it instead of the other way around?
 - What happened when you ran `npm run dev`? What is actually running on your machine right now?
-

Lesson 1 — JSX Fundamentals

Objective

Understand JSX rules by hand-building a static (non-interactive) todo list — no state yet, just hardcoded data.

Steps

6. In `App.jsx`, add a hardcoded array of todos above the component:

```
const initialTodos = [
  { id: 1, text: "Learn JSX", done: false },
  { id: 2, text: "Understand props", done: false },
  { id: 3, text: "Build a todo list", done: true },
];
```

7. Render them using `.map()` inside your JSX, remembering the `key` rule:

```
function App() {
  return (
    <div className="app">
      <h1>My Todo App</h1>
      <ul>
        {initialTodos.map((todo) => (
          <li key={todo.id}>
            {todo.text} — {todo.done ? "Done" : "Not done"}
          </li>
        ))}
      </ul>
    </div>
  );
}
```

8. Try (on purpose) removing the `key={todo.id}` and check your browser console. Read the warning. Put it back.
9. Try changing `className` to `class` on the `<div>`. Read the error/warning. Change it back.

Checkpoint

Your page shows a bulleted list of 3 todos with "Done"/"Not done" labels, matching the hardcoded data.

Reflection Questions

- Why does JSX require `className` instead of `class`? What is JSX actually being converted into behind the scenes?
 - What warning did you get when you removed `key`? In your own words, why does React need a `key` for list items?
 - Explain the ternary (`todo.done ? "Done" : "Not done"`) in plain English. Where else could you imagine using this pattern?
-

Lesson 2 — Components & Props

Objective

Break your single `App.jsx` file into reusable components that receive data through props.

Steps

10. Create `src/components/ToDoItem.jsx`:

```
function ToDoItem({ todo }) {
  return (
    <li>
      {todo.text} — {todo.done ? "Done" : "Not done"}
    </li>
  );
}

export default ToDoItem;
```

11. Create `src/components/ToDoList.jsx`:

```
import ToDoItem from "../ToDoItem";

function ToDoList({ todos }) {
  return (
    <ul>
      {todos.map((todo) => (
        <ToDoItem key={todo.id} todo={todo} />
      ))}
    </ul>
  );
}

export default ToDoList;
```

12. Update `App.jsx` to use it:

```
import ToDoList from "../components/ToDoList";

const initialTodos = [
  { id: 1, text: "Learn JSX", done: false },
  { id: 2, text: "Understand props", done: false },
  { id: 3, text: "Build a todo list", done: true },
];

function App() {
```

```
return (  
  <div className="app">  
    <h1>My Todo App</h1>  
    <TodoList todos={initialTodos} />  
  </div>  
);  
}  
  
export default App;
```

Checkpoint

The app looks exactly the same as Lesson 1 — but now the same list is built from three separate files working together.

Reflection Questions

- If the app *looks* identical to Lesson 1, what was the point of this lesson? What problem does splitting into components solve as an app grows?
 - In `TodoItem`, we wrote `function TodoItem({ todo })` instead of `function TodoItem(props)`. What is this syntax doing, and why might it be useful?
 - Props are described as "read-only." Try to change `todo.text` inside `TodoItem` directly and see what happens or what feels wrong about it. Why do you think React enforces this rule?
-

Lesson 3 — State with `useState`

Objective

Move from hardcoded data to data that can actually change, using React's `useState` hook.

Steps

13. In `App.jsx`, import `useState` and convert `initialTodos` into state:

```
import { useState } from "react";
import TodoList from "../components/TodoList";

const startingTodos = [
  { id: 1, text: "Learn JSX", done: false },
  { id: 2, text: "Understand props", done: false },
  { id: 3, text: "Build a todo list", done: true },
];

function App() {
  const [todos, setTodos] = useState(startingTodos);

  return (
    <div className="app">
      <h1>My Todo App</h1>
      <TodoList todos={todos} />
    </div>
  );
}

export default App;
```

14. Add a temporary test button to prove state works — add this above `<TodoList />`:

```
<button onClick={() => setTodos([...todos, { id: Date.now(), text: "New
item", done: false }])}>
  Add test item
</button>
```

15. Click the button several times. Watch new items appear without a page reload.

16. Remove the test button — you'll build a proper form for this in Lesson 4.

Checkpoint

Clicking "Add test item" instantly adds a new row to the list, with no page refresh.

Reflection Questions

- What is the difference between `todos` and `setTodos`? Why can't you just do `todos.push(...)` directly?
 - What does `[...todos, newItem]` mean? Why do we spread the old array instead of modifying it directly?
 - In plain terms: what does "React re-renders when state changes" actually mean, based on what you just observed?
-

Lesson 4 — Handling Events & Forms (Adding Todos)

Objective

Build a real input form so the user can type and add their own todos.

Steps

17. Create `src/components/AddTodoForm.jsx`:

```
import { useState } from "react";

function AddTodoForm({ onAdd }) {
  const [text, setText] = useState("");

  function handleSubmit(e) {
    e.preventDefault();
    if (text.trim() === "") return;
    onAdd(text);
    setText("");
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="text"
        value={text}
        onChange={(e) => setText(e.target.value)}
        placeholder="Add a new todo..."
      />
      <button type="submit">Add</button>
    </form>
  );
}

export default AddTodoForm;
```

18. In `App.jsx`, write the function that adds a todo and pass it down:

```
import AddTodoForm from "../components/AddTodoForm";

function App() {
  const [todos, setTodos] = useState(startingTodos);

  function handleAddTodo(text) {
    const newTodo = { id: Date.now(), text, done: false };
    setTodos([...todos, newTodo]);
  }
}
```

```
return (  
  <div className="app">  
    <h1>My Todo App</h1>  
    <AddTodoForm onAdd={handleAddTodo} />  
    <TodoList todos={todos} />  
  </div>  
);  
}
```

Checkpoint

Typing into the input and pressing Enter (or clicking Add) adds a new todo to the list, and clears the input afterward.

Reflection Questions

- `AddTodoForm` doesn't know anything about the `todos` array — it only calls `onAdd(text)`. Why is it designed this way instead of letting `AddTodoForm` update the list itself?
 - What does `value={text}` and `onChange={...}` together create? Look up the term "controlled component" and explain it in your own words.
 - Why do we call `e.preventDefault()` in `handleSubmit`? What happens if you remove it?
-

Lesson 5 — Updating State (Toggle Complete & Delete)

Objective

Learn to update and remove items from an array in state without mutating it directly.

Steps

19. In `App.jsx`, add two more handler functions:

```
function handleToggle(id) {
  setTodos(
    todos.map((todo) =>
      todo.id === id ? { ...todo, done: !todo.done } : todo
    )
  );
}

function handleDelete(id) {
  setTodos(todos.filter((todo) => todo.id !== id));
}
```

20. Pass these down through `TodoList` to `TodoItem`:

```
// TodoList.jsx
function TodoList({ todos, onToggle, onDelete }) {
  return (
    <ul>
      {todos.map((todo) => (
        <TodoItem key={todo.id} todo={todo} onToggle={onToggle}
onDelete={onDelete} />
      ))}
    </ul>
  );
}
```

21. Update `TodoItem.jsx` to use them:

```
function TodoItem({ todo, onToggle, onDelete }) {
  return (
    <li>
      <span
        style={{ textDecoration: todo.done ? "line-through" : "none",
cursor: "pointer" }}
        onClick={() => onToggle(todo.id)}
      >
        {todo.text}
      </span>
    </li>
  );
}
```

```
        </span>
        <button onClick={() => onDelete(todo.id)}>Delete</button>
      </li>
    );
  }
}
```

22. Wire it up in `App.jsx`:

```
<TodoList todos={todos} onToggle={handleToggle} onDelete={handleDelete}
/>
```

Checkpoint

Clicking a todo's text toggles a strikethrough style. Clicking "Delete" removes that item permanently from the list.

Reflection Questions

- In `handleToggle`, why do we use `.map()` and check `todo.id === id` instead of finding the todo and changing it directly?
 - What does `{ ...todo, done: !todo.done }` do? Why don't we write `todo.done = !todo.done` instead?
 - Props (`onToggle`, `onDelete`) are being passed down two levels (`App` → `TodoList` → `TodoItem`). This is sometimes called "prop drilling." What do you think might get annoying about this as an app grows larger?
-

Lesson 6 — Conditional Rendering (Filters & Empty States)

Objective

Show/hide content based on state — filtering todos and handling the "no todos" case gracefully.

Steps

23. Add a filter state in `App.jsx`:

```
const [filter, setFilter] = useState("all"); // "all" | "active" | "completed"

const visibleTodos = todos.filter((todo) => {
  if (filter === "active") return !todo.done;
  if (filter === "completed") return todo.done;
  return true;
});
```

24. Add filter buttons above `TodoList`:

```
<div className="filters">
  <button onClick={() => setFilter("all")}>All</button>
  <button onClick={() => setFilter("active")}>Active</button>
  <button onClick={() => setFilter("completed")}>Completed</button>
</div>
```

25. Pass `visibleTodos` instead of `todos` into `TodoList`.

26. Handle the empty state — inside `TodoList.jsx`:

```
function TodoList({ todos, onToggle, onDelete }) {
  if (todos.length === 0) {
    return <p>No todos here. Add one above!</p>;
  }
  return (
    <ul>
      {todos.map((todo) => (
        <TodoItem key={todo.id} todo={todo} onToggle={onToggle}
        onDelete={onDelete} />
      ))}
    </ul>
  );
}
```

Checkpoint

Clicking "Active" hides completed todos. Clicking "Completed" shows only done ones. If a filter results in zero items, you see the "No todos here" message instead of an empty list.

Reflection Questions

- `visibleTodos` is a new array computed from `todos` every time the component renders — it is NOT stored in its own `useState`. Why doesn't the filtered list need its own state variable?
 - What would happen visually if you forgot to handle the `todos.length === 0` case? Why does this small detail matter for a real app's user experience?
 - Where else in a real app (not just a todo list) can you imagine needing this "if empty, show a message instead" pattern?
-

Lesson 7 — Multiple Pages with React Router

Objective

Turn your single-page app into a multi-page app using React Router, without full page reloads.

Steps

27. Install React Router:

```
npm install react-router-dom
```

28. Create `src/pages/Home.jsx` — move your existing todo app content here (form + filters + list). It should now accept the state as props, OR (simpler for this lesson) keep the state at the `App` level and pass it down as props into `Home`.

29. Create two simple additional pages:

`src/pages/About.jsx`:

```
function About() {  
  return (  
    <div>  
      <h2>About This App</h2>  
      <p>A todo app built while learning React fundamentals.</p>  
    </div>  
  );  
}  
export default About;
```

`src/pages/Stats.jsx`:

```
function Stats({ todos }) {  
  const completed = todos.filter((t) => t.done).length;  
  return (  
    <div>  
      <h2>Stats</h2>  
      <p>Total todos: {todos.length}</p>  
      <p>Completed: {completed}</p>  
      <p>Remaining: {todos.length - completed}</p>  
    </div>  
  );  
}  
export default Stats;
```

30. Rebuild `App.jsx` to hold the routes and the shared state:

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import { useState } from "react";
import Home from "../pages/Home";
import About from "../pages/About";
import Stats from "../pages/Stats";

function App() {
  const [todos, setTodos] = useState(startingTodos);
  // ...keep handleAddTodo, handleToggle, handleDelete here...

  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/stats">Stats</Link>
        <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route
          path="/"
          element={
            <Home
              todos={todos}
              onAdd={handleAddTodo}
              onToggle={handleToggle}
              onDelete={handleDelete}
            />
          }
        />
        <Route path="/stats" element={<Stats todos={todos} />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Checkpoint

Clicking the nav links switches between Home, Stats, and About instantly, with no full-page reload (watch the browser's loading spinner — it shouldn't spin). The Stats page correctly counts your todos.

Reflection Questions

- Why does `App.jsx` still hold the `todos` state, instead of moving it into `Home.jsx`? What would break if `Stats.jsx` needed its own separate copy of the todos?

- What is the actual difference between clicking a `<Link>` and clicking a normal `<a href= "... ">` tag? (Try temporarily replacing one `<Link>` with an `<a>` tag and compare what happens.)
 - In your own words, what problem does React Router solve that plain HTML links can't?
-

Lesson 8 — Persisting Data with `useEffect` and `localStorage`

Objective

Make todos survive a page refresh — your first taste of a "side effect" in React.

Steps

31. In `App.jsx`, import `useEffect` and change how state is initialized:

```
import { useState, useEffect } from "react";

const [todos, setTodos] = useState(() => {
  const saved = localStorage.getItem("todos");
  return saved ? JSON.parse(saved) : startingTodos;
});

useEffect(() => {
  localStorage.setItem("todos", JSON.stringify(todos));
}, [todos]);
```

32. Refresh the page after adding/completing/deleting some todos. Confirm they're still there.

33. Open your browser DevTools → Application tab → Local Storage, and inspect the saved data.

Checkpoint

After adding and completing a few todos, refreshing the browser (F5) does NOT reset your list — it looks exactly as you left it.

Reflection Questions

- What is the array `[todos]` at the end of `useEffect(..., [todos])` doing? What do you think would happen if you changed it to `[]`, or removed it entirely?
 - `localStorage` only stores strings, which is why we use `JSON.stringify` and `JSON.parse`. Explain in your own words why an object like `{ id: 1, text: "Learn JSX", done: false }` can't be stored directly.
 - This is your first "side effect" — something that reaches **outside** of React (into the browser's storage). Why do you think React makes you use a special hook (`useEffect`) for this instead of just running the code directly in the component body?
-

Final Project Checklist

Before submitting, confirm your app can:

- ☐ Add a new todo via a form
- ☐ Toggle a todo complete/incomplete
- ☐ Delete a todo
- ☐ Filter by All / Active / Completed
- ☐ Show an empty-state message when a filter has no results
- ☐ Navigate between Home / Stats / About without page reloads
- ☐ Keep todos after refreshing the browser
- ☐ Use at least: `useState`, `useEffect`, props, conditional rendering, `.map()` with keys, and React Router

Reflection Template

Copy this for each lesson in your yellow pad:

```
## Lesson [number] - [topic]
Date:

What I built:

What I understood, in my own words:

What confused me / what I had to look up:
```